

✦ JOSA //

PR Etiquette & Maintainer Friendly Contributions

/* Foundations refreshed and elevated */

OpenLab Apprenticeship 2026

Week 02

// 1 //

Learning Goals

- Read and follow a project's [CONTRIBUTING.md](#) and issue templates as a discipline, not as a formality.
- Write PR descriptions that get merged faster.
- Understand and apply **Conventional Comments** when reviewing or being reviewed.
- Develop a contribution-search workflow that finds high-signal issues fast.

// 2 //

Technical Deep Dive

// 2.1 Anatomy of a PR maintainers love

A merge-friendly PR has all of these:

Title: `fix(parser): prevent stack overflow on deeply nested input`

Body:

What

Caps recursion depth in `parseExpression()` at 1000 levels and returns a structured error instead of overflowing.

Why

Issue #1284: A user-supplied JSON file with 50,000 levels of nesting crashes the CLI. Fuzz-tested input in the linked reproduction.

How

- Added a `depth` parameter threaded through the parser.
- New `MaxDepthExceededError` (subclass of `ParseError`).
- Limit is configurable via `parse(input, { maxDepth })`.

Test

- Unit test in `parser.test.ts` reproduces the original crash.
- New test covers the configurable-limit path.
- Manually re-ran the OSS-Fuzz corpus — no regressions.

```
## Notes for reviewers
```

```
The `depth++` placement is intentional: incrementing *before*  
the recursive call guarantees we never enter the recursion when  
already at the limit.
```

```
Closes #1284
```

// 2.2 Key properties of a great PR

- **Title is a Conventional Commit.** The maintainer can copy-paste it as the squash-merge commit message.
- **What / Why / How / Test** is a four-section template that surfaces everything a reviewer needs.
- **Closes #1284** auto-links and auto-closes the issue when merged. Other valid keywords: [Fixes](#), [Resolves](#).
- **A Notes for reviewers section** flags non-obvious decisions before someone has to ask.
- **Small.** ~100 lines of diff, one logical change.

// 2.3 Reading CONTRIBUTING.md as a real signal

When you find an issue you want to work on, before writing code:

1. Read [CONTRIBUTING.md](#). It tells you whether to comment “I’ll take this” (assignment culture) or just open a PR (do-ocracy culture). Misreading this is the single most common rookie mistake. It also tells you whether the project requires sign-offs, DCO (Developer Certificate of Origin), or CLA (Contributor License Agreement), how to run tests locally, and the style conventions and linters/code formatters.
2. Read recent merged PRs of a similar size. They show the actual cultural norms – what the README says vs. what reviewers actually accept.
3. Read the project’s [CODE_OF_CONDUCT.md](#). Yes, really.

// 2.4 Conventional Comments

When you start reviewing other people’s code (Week 4), and to give better feedback even now, adopt [Conventional Comments](#). Every comment is prefixed with a label and an optional decoration:

suggestion: Extract this into a helper — it's used 3 places now.

issue (non-blocking): This swallows the original exception. Prefer
`raise X from e` to keep the chain.

praise: Beautiful refactor — much easier to follow than the original.

nitpick: Trailing whitespace on line 42.

question: Is there a reason we're not memoizing this?

The labels — [praise](#), [nitpick](#), [suggestion](#), [issue](#), [question](#), [thought](#), [chore](#), [note](#) — separate **what is required to merge** from what is style preference, curiosity, or appreciation. Decorations like [\(non-blocking\)](#) and [\(if-minor\)](#) further reduce friction. Both authors and reviewers benefit: nobody has to guess severity.

// 3 //

Learning Material

Topic	Resource
The canonical guide to contributing	Open Source Guides – How to Contribute
Writing review-friendly PRs	Google eng-practices: CL Author's Guide
Comment labeling spec	Conventional Comments
PR description tactics	How to Write a Pull Request Description (FreeCodeCamp)
Searching GitHub well	GitHub – Searching issues and pull requests
Finding contribution-friendly projects	Up For Grabs and Good First Issues

// 4 //////////////////////////////////

Suggested Repositories

- [tldr-pages/tldr](#) – Strict but clear contribution guidelines, fast review cycle, very high traffic.
- [httpie/cli](#) – Approachable Python codebase, well-curated issues, real maintainer engagement.
- [t3-oss/create-t3-app](#) – Modern TypeScript, explicit [unconfirmed bug](#) and [good first issue](#) labels.
- [github/docs](#) – Excellent docs PR pipeline reviewed by GitHub staff.
- [freeCodeCamp/freeCodeCamp](#) – Massive curriculum repo; many [help wanted](#) issues teach process.

// 5 //////////////////////////////////

Your Tasks for the Week

1. **Two well-formed PRs (docs Improvement OR small bug fixes).** Open two PRs (in any repo) that follow the What/Why/How/Test template and use a Conventional Commits title. They can be small – a typo fix qualifies – but the PR description must be complete.
2. **A practice review.** Go to any active, reasonably small open PR in any of the suggested repos and write at least 2 review comments. (You don't need merge rights to leave review comments; GitHub allows anyone to review.)

Soft Skill of the Week: Asking Good Questions

When you get stuck — and you will — *how* you ask for help determines whether senior people want to keep helping you. The classic resource is Eric Raymond's [How to Ask Questions the Smart Way](#); read it for the principles, but be aware its tone is dated and at times harsh. The principles still hold.

// 6.1 The Condensed Version

1. **Search first.** Read the docs, search the issue tracker, search Stack Overflow. If you find a related thread, link to it in your question and explain why it didn't solve your problem. This shows respect for the responder's time.
2. **State the goal, not just the obstacle.** "I'm trying to deploy a static site to Cloudflare Pages and the build fails with X" is much better than "How do I fix error X?" — because the responder may suggest a different approach entirely.
3. **Provide a Minimum Reproducible Example.** (We'll go deep on this in Week 3.) Code, exact commands, exact error output, environment info (OS, language version, package versions).
4. **Show what you've tried.** Three things, ideally. Even if they're all wrong, they show effort and rule things out.
5. **Don't ask if you can ask. Just ask.** "Can someone help me with X?" wastes a round-trip. Lead with the question.



TIP

A useful framing: write your question imagining the responder is doing you a favor on their unpaid time. (Because, in OSS, they are.)

// 6.2 The Condensed Version

- [How to Ask Questions the Smart Way — ESR](#)
- [Stack Overflow — How do I ask a good question?](#)